Desenvolvimento e Testes

Como o Movecity foi construído, testado e entregue

Relatório de Construção e Testes — Movecity

> **Versão 1.0 · 30/10/2026.** Descreve a construção e os testes do Movecity **como estão hoje**. O código descrito é o de `origin/homolog` (ambiente publicado); os testes manuais foram executados em produção em 27 e 28/10/2026; os testes automatizados foram contados no log do CI de 28/10/2026. O que não foi medido ou não foi feito está dito como tal.

Resumo

O Movecity é um aplicativo web em Next.js, com um Gateway e três serviços FastAPI (autenticação, mobilidade e colaboração) no Fly.io, banco PostgreSQL no Neon e frontend na Vercel. Todo o código foi gerado por agentes de IA, conduzidos pelos seis integrantes do grupo, dentro de um processo com issue, BDD, PR, CI e revisão.

Medida	Resultado
Testes automatizados	1.640, todos passando no CI (844 de backend e 796 de frontend)
Cobertura	93,7% no backend e 89,1% no frontend (medida em 16/10)
Casos manuais em produção	49 executados: 32 aprovados, 11 aprovados em parte ou com ressalva, 1 reprovado por configuração, 3 não verificáveis fora de campo e 2 pendentes
Defeitos catalogados	26, dos quais 10 de conexão entre as peças e 16 de regra, dado ou interface
Usabilidade	Avaliação heurística de Nielsen por um avaliador, de forma agêntica

Onde está cada coisa. Este documento traz a construção, o projeto de interface, os testes e o processo de desenvolvimento. Os detalhes de teste (plano, casos, execução com as evidências, matriz de rastreabilidade, testes unitários, cobertura e defeitos, integração e usabilidade) estão no documento de **Anexos de testes**, com um índice no início.

1. Introdução

1.1 Propósito e escopo

Este documento reúne as partes do roteiro de software da disciplina Projeto de Solução Computacional que não estão no Documento de Visão, no DRS nem no Documento de Arquitetura: o projeto da interface com o usuário (2.1), a construção (3) e os testes (4). Ele traz as métricas extraídas do repositório, do CI e da execução dos casos de teste, cada uma com a sua fonte e a sua data.

1.2 Onde está cada item do roteiro

Item do roteiro	Onde está
1.1 Visão e escopo	Documento de Visão (`docs/documento_visao.md`)
1.2 Especificação de requisitos	DRS (`docs/drs.md`)
2.1 Projeto de interface com o usuário	Seção 2 deste relatório e protótipo de alta fidelidade (`prototipo-alta-fid/`)
2.2 Arquitetura lógica	Documento de Arquitetura, seção 4 (Visão Lógica)
2.3 Arquitetura física	Documento de Arquitetura, seção 7 (Visão de Implantação)
2.4 Projeto de banco de dados	Documento de Arquitetura, seção 8 (Visão de Dados, com o MER) e seção 3.4 deste relatório

Item do roteiro	Onde está
3.1 Código-fonte	Seção 3.1 deste relatório e o repositório
3.2 Teste de unidade	Seção 3.2
3.3 Interfaces com o usuário	Seção 3.3
3.4 Banco de dados	Seção 3.4
4.1 Teste funcional	Seção 4.1
4.2 Teste de integração	Seção 4.2
4.3 Teste de usabilidade (heurísticas de Nielsen)	Seção 4.3

1.3 Fontes e datas

Medida	Fonte	Data
Linhas e arquivos de código	`git grep -c ''` em `origin/homolog`	28/10/2026
Issues, PRs e commits	`gh issue list`, `gh pr list` e `git rev-list`	28/10/2026
Testes automatizados	log do CI, execução `40689632241`, commit `48483be`	28/10/2026
Cobertura de testes	`pytest --cov` e `jest --coverage` (`07c9edc`)	16/10/2026
Histórico do CI	`gh run list`	16/10/2026
Casos de teste manuais	`docs/qa/execucao-casos-p1.md`	27 e 28/10/2026
Defeitos	`docs/qa/relatorio-cobertura-e-defeitos.md`	28/10/2026
Usabilidade	`docs/qa/avaliacao-usabilidade-nielsen.md`	27 e 28/10/2026

2. Projeto de interface com o usuário

2.1 Protótipo de alta fidelidade

A interface foi projetada em um protótipo de alta fidelidade ("MoveCity App v3"), versionado em `prototipo-alta-fid/` (páginas HTML estáticas, uma por tela) e publicado no GitHub Pages pelo workflow `deploy-pages.yml`. O protótipo define a linguagem visual (cores, componentes, ícones, espaçamento) e o fluxo em largura de celular. Ele **não define** o comportamento em tablet e desktop: essa decisão foi do grupo (`docs/adequacao-ui-prototipo-v3.md`).

2.2 Decisões de interface

Decisão	Motivo e fonte
Mapa como tela principal, com busca de linha, planejador de rota e parada no mesmo painel	Mantém a ação principal (saber onde está o ônibus) a um toque; fluxo do protótipo
Painel único com três alturas no celular (rota, linha e parada)	A US #217 trocou painéis diferentes por uma folha que sobe e desce; **não foi conferida em aparelho real**
Todo tempo exibido diz a origem: "ao vivo", "previsto" ou "Sem previsão no momento"	Evita número sem fonte (RN-010 do DRS; US #19, #159 e #173)
Faixa "Em viagem" fixa no topo, e aviso de proximidade por cima de qualquer tela	Quem está em viagem precisa ser avisado em qualquer tela (US #171, #172)
Mapa com base CARTO Voyager e recuo para o tile padrão do OpenStreetMap	Legibilidade de linha, parada e veículo; ver ADR no Documento de Arquitetura

Decisão	Motivo e fonte
Aplicativo instalável (PWA) com *service worker* que guarda só a casca	Receber o Web Push; dado de transporte nunca é cacheado
Tutorial de 4 etapas no primeiro acesso	US #30

3. Construção

3.1 Código-fonte

Repositório e processo. O código está num repositório único no GitHub, com gitflow (`feat/` e `fix/` passam por `homolog` antes de `main`; `docs/` e `chore/` vão direto para `main`), Conventional Commits, CI obrigatório e PR ligado a uma issue. **Todo o código foi gerado por agentes de IA, conduzidos por seis estudantes que especificaram (issue, cenários BDD, diagrama de sequência), revisaram e aprovaram.** O ciclo completo está em `docs/manual-sprint.md`.

Tamanho. Contagem em `origin/homolog`, em 28/10/2026 (inclui comentários e linhas em branco):

Parte	Arquivos	Linhas
Backend (Python): total	179	26.590
código de produção	112	13.243
testes	67	13.347
Frontend (TypeScript e TSX): total	94	22.877
código de produção	51	10.593
testes	43	12.284

Estrutura.

Componente	Conteúdo
`src/backend/gateway`	API Gateway: valida o JWT do cookie, faz proxy e grava os cookies
`src/backend/auth`	Cadastro, login por e-mail e Google, sessões, renovação, recuperação de senha por e-mail
`src/backend/mobilidade`	Linhas, paradas, posição ao vivo, ETA, rota, viagem, alerta de proximidade, Web Push, ingestão da SEMOB
`src/backend/colaboracao`	Ocorrências e confirmações, rotas favoritas, preferências de notificação
`src/backend/shared`	Acesso ao banco e configuração comuns
`src/frontend`	Aplicativo Next.js (App Router), 13 telas, PWA
`.github/workflows`	5 workflows: `ci.yml`, `deploy-fly.yml`, `deploy-vercel.yml`, `deploy-pages.yml`, `ingestao-semob.yml`

O backend declara **68 rotas** (decoradores `@router` e `@app`), em 4 aplicações FastAPI independentes no Fly.io (gateway, auth, mobilidade, colaboração).

Volume do trabalho (28/10/2026).

Medida	Valor	Observação
Issues	103	`gh issue list --state all`
Pull Requests	152	`gh pr list --state all`

Medida	Valor	Observação
Commits em `homolog`	336	inclui commits de merge
Período	15/08 a 28/10/2026	cerca de 10 semanas

3.2 Teste de unidade

Ferramentas. `pytest` no backend (com PostgreSQL 16 e as migrations aplicadas no CI) e Jest no frontend. Todo PR roda a suíte no CI.

Resultado.

Suíte	Testes	Passando	Falhando	Fonte
Backend (pytest)	844	844	0	CI de 28/10
Frontend (Jest, 43 suítes)	796	796	0	CI de 28/10
Total	**1.640**	**1.640**	**0**	execução `40689632241`

Em 16/10 eram 1.260 testes (671 de backend em 56 arquivos e 589 de frontend em 33 suítes); o registro original da #151 tinha 297. O aumento de 380 testes (173 no backend e 207 no frontend) é o que entrou na `homolog` entre 16/10 e 28/10.

Disciplina. Desde 22/09, 27 de 27 PRs `feat` e `fix` trouxeram arquivo de teste (medido em 16/10).

Cobertura (medida em 16/10, `07c9edc`; **não foi remedida**, porque o CI não mede cobertura):

Serviço (backend)	Instruções de produção	Não cobertas	Cobertura
mobilidade	2.700	146	94,6%
colaboracao	535	35	93,5%
auth	372	29	92,2%
gateway	180	31	82,8%
shared	41	0	100,0%
Código de produção	**3.828**	**241**	**93,7%**

No frontend, a cobertura de instruções é **89,07%** (1.622 de 1.821), de ramos 84,65% e de funções 86,09%. **Ressalva:** o Jest do projeto não define `collectCoverageFrom`, então só entram na conta os arquivos importados por algum teste; 3 arquivos ficam fora da medição, e o número é um piso otimista.

Os arquivos com menor cobertura no backend são o provider real do Google Maps (30%, chamada externa), o worker de monitoramento (62%), a montagem da aplicação de mobilidade (73%) e as rotas e o proxy do Gateway (81% e 83%, testados com `httpx` simulado).

Limite. Os testes de unidade usam simulação (*mock*) nas pontas externas e nos serviços vizinhos; por isso não pegam defeitos de conexão (ver 4.2).

3.3 Interfaces com o usuário

13 telas implementadas no frontend (`src/frontend/app`):

Tela	Rota	US principais
Mapa (busca de linha, planejador de rota, parada, viagem)	`/mapa`	#14 a #21, #171, #172, #190
Ocorrências e reporte	`/ocorrencias`, `/ocorrencias/reportar`	#23, #24, #26
Rotas salvas	`/favoritos`	#25
Alertas (preferências de notificação)	`/alertas`	#29
Cadastro, login e recuperação de senha	`/cadastro`, `/login`, `/recuperar-senha`, `/redefinir-senha`	#10, #11, #12, #13
Configurações e tutorial	`/configuracoes`, `/tutorial`	#30
Página inicial e status	`/`, `/status`	apoio

O aviso de proximidade da viagem (#172) existe dentro do aplicativo e por Web Push; o grupo testou o Web Push em iPhone, com resultado funcional.

3.4 Banco de dados

PostgreSQL no Neon (serverless), **sem PostGIS**: a geometria fica em colunas JSON e a única operação espacial (associar paradas ao traçado) roda uma vez por mês, na ingestão, em Python.

Medida	Valor
Tabelas (`__tablename__` nos modelos)	21, sendo 4 de teste sem função no produto, criadas pela migration inicial
Migrations (Alembic)	20
Serviços que acessam o banco	auth, mobilidade e colaboração (o Gateway não acessa)

Tabelas do domínio, por serviço:

Serviço	Tabelas
auth	`usuarios`, `sessoes`, `tokens_reset_senha`
mobilidade	`linha`, `rota`, `rota_celula`, `parada`, `rota_parada`, `viagem`, `alertas_proximidade`, `inscricao_push`, `alertas`, `linhas_acompanhadas`
colaboracao	`ocorrencia`, `confirmacoes_ocorrencia`, `rotas_favoritas`, `preferencias_notificacao`

Integridade. Dentro de um serviço há chaves estrangeiras no banco (por exemplo `sessoes.usuario_id`). Entre serviços, o ORM não declara a chave para `usuarios`; em `rotas_favoritas` e `preferencias_notificacao` a chave existe só na migration (defeito 20 e ADR-22). A viagem **não guarda posição do usuário** (LGPD, ADR-15). O MER completo está no Documento de Arquitetura, seção 8.

4. Testes

4.1 Teste funcional

Método. 49 casos manuais escritos a partir dos cenários BDD das USs entregues, executados **em produção** (`movecity-frontend.vercel.app` e o Gateway), pelo navegador e por chamadas à API, em desktop (1366 px) e celular (390 px), em 27 e 28/10/2026. Cada caso tem pré-condição, passos, resultado esperado, severidade e prioridade (18 de prioridade P1, 24 de P2 e 7 de P3). As evidências são **59 imagens** em `docs/qa/evidencias/`.

Casos por área: mapa 9, viagem 8, autenticação 7, rota 7, colaboração 5, parada 4, ETA 4, segurança 3, notificações 2.

Resultado.

Estado	Casos	Quais
Aprovado	32	a lista completa está em `docs/qa/execucao-casos-p1.md`
Aprovado em parte ou com ressalva	11	AUT-01, AUT-03, AUT-05, COL-04, COL-05, ETA-04, PAR-03, ROT-06, ROT-07, SEG-01, VIA-02
Reprovado por configuração	1	VIA-01
Não verificável hoje	3	NOT-02, VIA-07, VIA-08
Pendente	2	MAP-03, ETA-03

Por que não fecharam.

Caso	Motivo
VIA-01	O embarque é aceito longe do ônibus porque o raio de embarque está em 100 km em produção, por decisão do grupo só para demonstrar o "Peguei este ônibus" sem estar dentro de um ônibus. A regra de uso real é 100 m.
VIA-07	A regra "ônibus chegando" exige ônibus a 2 min de uma parada a até 400 m; com o raio em 100 km ela fica mascarada. Só se prova em campo e com o raio em 100 m.
VIA-08	Exige viagem com destino e um ônibus se aproximando da parada de descida; só se prova em campo.
NOT-02	O alerta de atraso roda no servidor e não tem canal de entrega ao usuário. A lógica é coberta por testes unitários.
MAP-03, ETA-03	Exigem localização simulada pelo DevTools (Sensors); a ferramenta de automação perde o controle com o DevTools aberto, e a execução é manual.
AUT-05	E-mail, link e nova senha foram comprovados; faltam, na tela, o link já usado e o expirado (tratados pelo código).
COL-05	A validade da ocorrência é por tipo (de 15 a 120 min), não 60 min; a conferência foi feita uma vez, cerca de 64 min depois.

Resultado da primeira execução dos 18 casos P1: 4 reprovaram (AUT-01, AUT-03, PAR-01 e PAR-04) e viraram defeitos; depois da correção passaram na revalidação.

Correções validadas em produção durante a execução: cadastro sem aceite dos termos (#211), rota de refresh e renovação automática da sessão (#212 e #221; a rede mostrou 401, `POST /auth/refresh` 200 e a consulta refeita com 200), tela de alertas (#29, #213), horários de saída já passados (#215), "Parada sem nome" (#232) e coordenadas na URL (#231). O e-mail de recuperação de senha por SMTP (#237) foi entregue e a nova senha definida pelo link.

4.2 Teste de integração

O que o CI cobre. Em todo PR: o `uvicorn` sobe o Gateway e exige `GET /health`; as migrations rodam num PostgreSQL de verdade; o `pytest` roda contra esse banco; e, para cada um dos 4 serviços, uma instalação limpa com **só** o `requirements.txt` do serviço importa a aplicação, como o contêiner faz no boot. Também rodam o Jest e a validação de título de PR e commits (7 verificações no total).

O que não cobre. A conversa entre as peças: o proxy do Gateway é testado com `httpx` simulado, e o frontend chama o Gateway só em testes com `fetch` simulado. 2 arquivos de teste usam SQLite em memória em vez do PostgreSQL. Não há teste automatizado de ponta a ponta (E2E) nem de carga (fora de escopo).

O que cobriu essa lacuna. A execução manual dos 49 casos em produção atravessa o caminho completo (navegador, Gateway, serviços, banco). Foi ela que achou, entre outros, a rota `/auth/refresh` inexistente, o erro 500 das preferências e a sessão expirada tratada como "nenhum ônibus".

Defeitos de integração. Dos 26 defeitos catalogados, 10 são de conexão entre as peças (dependência faltando no serviço, contrato front ↔ Gateway, ordem de middleware, rotas públicas, cabeçalhos duplicados, import perdido num merge, chave estrangeira sem o modelo carregado no serviço isolado). É a prioridade do plano do relatório de integração (contrato front ↔ Gateway e teste do Gateway com HTTP real), **ainda não executado**.

Histórico do CI de PR (`gh run list`, medido em 16/10):

Janela	Execuções concluídas	Verdes	% verde
Desde 18/06	199	148	74%
Sprint 2	76	60	79%
Sprint 3 (desde 05/10)	35	33	94% (mediana de 1,1 min)

4.3 Teste de usabilidade (heurísticas de Nielsen)

Método e limites. Avaliação heurística da interface em produção, pelas 10 heurísticas de Nielsen, com severidade de 0 a 4, durante a execução dos casos. Foi feita por **um único avaliador, de forma agêntica** (um agente de IA conduzido por um integrante do grupo), por decisão do grupo. A técnica pede 3 a 5 avaliadores independentes, e um agente não tem a percepção de um passageiro. **Não houve teste com usuários reais.** Os achados são hipóteses de usabilidade com evidência de tela.

#	Heurística	Severidade	Principal achado
1	Visibilidade do estado do sistema	2	O selo "ao vivo" já conviveu com posição de 2 min atrás
2	Correspondência com o mundo real	3	"Parada sem nome" (#232, corrigida)
3	Controle e liberdade do usuário	2	Sem como desfazer ou apagar um reporte enviado
4	Consistência e padrões	2	Painel de linha diferente da folha de rota; controle de usuário duplicado (corrigido)
5	Prevenção de erros	2	Cadastro sem termos (corrigido); "Buscar parada" só dispara com Enter (#236)
6	Reconhecimento em vez de memorização	3	Parada sem referência de onde fica
7	Flexibilidade e eficiência de uso	2	Coordenadas na URL ao tocar em "Ir" (corrigido, #231)
8	Estética e design minimalista	2	Painel da parada junta chegadas e saídas do terminal
9	Ajudar a reconhecer, diagnosticar e corrigir erros	3	Sessão expirada mostrada como "nenhum ônibus" (corrigido); erro de consulta confundido com "sem linha" (#233)
10	Ajuda e documentação	3	Telas mandavam "verificar o e-mail" sem e-mail (#230, corrigido)

Quatro heurísticas ficaram com severidade 3 (2, 6, 9 e 10); as demais, com 2. **Sem cobertura:** acessibilidade (leitor de tela, contraste medido, tamanho de toque), conexão lenta e usuários reais.

5. Defeitos e métricas

5.1 Defeitos

26 defeitos catalogados (13 até 16/10 e 13 achados em 27 e 28/10), em `docs/qa/relatorio-cobertura-e-defeitos.md`, cada um com issue ou PR, causa e forma de detecção.

Detectado por	Quantidade
Produção, antes do usuário final reportar	4
CI, antes do merge	2
Teste manual, execução de casos ou uso com dados reais	14
Auditoria, medição ou conferência de documento ou código	5
Não registrado	1

Natureza	Quantidade
Conexão entre as peças	10
Regra, dado ou interface	16

Abertos hoje: #216 (embarque, configuração do raio), #233 (rota "sem linha" uma vez), #234 (build depende do Google Fonts), #236 (dica do "Buscar parada"), #243 (mensagens sem acento), e #249 (texto "2 confirmações" na tela, servidor exige 3).

5.2 Quadro de métricas

Métrica	Valor	Data
Linhas de código (produção)	23.836 (13.243 backend, 10.593 frontend)	28/10
Linhas de teste	25.631 (13.347 backend, 12.284 frontend)	28/10
Rotas do backend	68	28/10
Tabelas e migrations	21 e 20	28/10
Telas	13	28/10
Issues, PRs e commits	103, 155 e 283	29/10
Testes automatizados	1.640 (844 e 796)	28/10
Cobertura (backend, frontend)	93,7% e 89,07%	16/10
PRs `feat` e `fix` com teste	41 de 42 desde 22/09	29/10
CI de PR verde, Sprint 3	94%	16/10
Casos manuais	49 de 49 executados (32 aprovados, 11 com ressalva, 1 reprovado por configuração, 3 não verificáveis, 2 pendentes)	28/10

Métrica	Valor	Data
Evidências	61 imagens	29/10
Defeitos catalogados	26	28/10
Usabilidade	10 heurísticas, 1 avaliador, 4 com severidade 3	28/10

6. Limites

Limite	Situação
Sem ambiente de homologação separado	`homolog` publica na produção da Vercel e o backend compartilha apps e banco; os casos que gravam dados afetam o ambiente visível (decisão de custo)
Sem teste de ponta a ponta automatizado	A integração ficou coberta pela execução manual em produção
Cobertura não medida no CI	Os números são de 16/10; o Jest mede só os arquivos importados
Sem validação em campo	Tolerância de ETA (2 a 10 min), regra "ônibus chegando" e aviso de proximidade em aparelho real
Raio de embarque em 100 km	Só para a demonstração; voltar a 100 m e validar a regra do ônibus chegando depois da defesa
Usabilidade com um avaliador, de forma agêntica	Sem usuários reais

7. Referências

- Documento de **Anexos de testes**: plano, casos, execução, matriz, unitários, cobertura e defeitos, integração e Nielsen.
- Documentos de Visão, DRS e Arquitetura.
- Issues #151 (estratégia de testes) e #204 (documentação final).

Processo de Desenvolvimento — Movecity

Como o Movecity foi construído: do item de backlog ao código em produção. Este capítulo reúne o que está nos guias do repositório (manual de sprint, workflow passo a passo, guia de contribuição, checklist de pré-commit e boas práticas de uso de IA) em um só texto.

1. Metodologia

O projeto combina duas abordagens, cada uma resolvendo um problema:

- **Lean Inception** (CAROLI, 2018), na fase de especificação, antes de qualquer código: personas, benchmarking e declaração de visão. Resolve o problema de construir a coisa certa.
- **Agentic Workflow**: o desenvolvimento é conduzido por agentes de IA (Claude Code e OpenCode) operando em nome de cada integrante, seguindo as regras do repositório. Resolve o problema de construir rápido sem perder rastreabilidade.

Todo o código foi gerado por agentes de IA, conduzidos pelos seis integrantes do grupo, que especificaram, revisaram e aprovaram. O que a IA escreve entra no repositório como trabalho de quem conduziu a sessão; commits, PRs e issues não levam assinatura de ferramenta.

2. Do backlog à issue

Toda User Story é formalizada como issue no GitHub, em três camadas separadas:

Camada	O que contém	Onde fica
User Story	`Como / Quero / Para` e cenários BDD (`Dado / Quando / Então`): o valor para o usuário	Corpo da issue
PRD	Objetivos de negócio, mudanças de interface e regras de negócio	Comentário
SPEC	Arquitetura, modelo de dados, endpoints e requisitos de desempenho	Comentário

A separação existe porque cada camada muda num ritmo: a US raramente muda, e a SPEC pode mudar várias vezes durante o desenvolvimento sem reescrever o valor de negócio.

Definição de pronto para desenvolver (DoR). Uma US só entra em desenvolvimento quando tem PRD e SPEC, critérios de aceite em BDD, estimativa de complexidade, dependências mapeadas e diagrama de sequência. O diagrama é a especificação técnica de execução: mostra, seta por seta, como Gateway, serviços e modelos interagem, e garante que duas pessoas implementando USs diferentes sigam o mesmo padrão (por exemplo, o JWT é validado sempre no Gateway, nunca duplicado no serviço de domínio).

3. Planejamento da sprint

- O time se reúne, revisa o backlog da sprint e cada integrante pega **uma US por sprint**. A regra evita que quem revisa vire gargalo e mantém o escopo dentro do prazo.
- Cada US vira um card no GitHub Projects, com checklist de desenvolvimento, revisão e deploy.
- A responsabilidade por épico é fixa (ver a distribuição de US no Documento de Requisitos), o que reduz troca de contexto.
- O trabalho foi organizado em cinco sprints; a última é a de fechamento e apresentação.

4. Ciclo de desenvolvimento com o agente

Para cada US, o agente segue esta ordem, e cada passo existe por um motivo:

1. **Ler a US e o diagrama antes de codar.** Sem isso, o código tende a divergir da arquitetura e a divergência só aparece na revisão, tarde demais.
2. **Criar a branch** a partir da base certa (ver o fluxo de branches abaixo).
3. **Implementar e testar localmente.** O CI roda os mesmos testes; falhar lá só gasta tempo.
4. **Antes do commit**, rodar o `scripts/verificar-deploy.sh`, que reproduz a verificação de contêiner do CI (cada serviço instala só o próprio `requirements.txt` e importa a aplicação) e confere o health check do Gateway, as rotas públicas e a rota protegida sem sessão. O motivo: uma dependência faltando no `requirements.txt` de um serviço passa no CI da raiz e derruba o contêiner no boot.
5. **Commit atômico, em Conventional Commits** (`feat`, `fix`, `docs`, `chore`, `test`...), o que deixa o histórico legível e permite reverter uma mudança específica. Mudanças em `docs/` e `src/` vão em commits separados.
6. **Abrir o PR** vinculado a uma issue (`Closes #N`), acompanhar o CI e aguardar a aprovação formal.
7. **Mergear, acompanhar o deploy e validar** o endpoint ou a tela real.

5. Fluxo de branches e revisão

```
feat/* e fix/*   →  homolog  →  main
docs/* e chore/* →  main (direto)
```

`feat` e `fix` mudam comportamento do sistema; por isso passam pela `homolog` antes de chegar à `main`. `docs` e `chore` não alteram o que roda em produção e vão direto para a `main`.

A `homolog` não é um ambiente isolado: o push nela já publica o frontend e o backend na produção pública, que compartilham os mesmos apps do Fly.io e o mesmo banco Neon. A decisão foi de custo, e o efeito prático é que **o merge em `homolog` equivale a publicar** e deve ser validado logo em seguida.

`main` e `homolog` são protegidas: exigem **uma aprovação formal** e CI verde. Só conta o botão **Approve** da aba "Files changed"; um comentário de texto não satisfaz a proteção. O autor do PR não pode aprovar o próprio, de modo que os PRs do orquestrador dependem de outro revisor ou de um bypass de administrador, que precisa ser decidido pelo orquestrador e registrado na issue relevante.

6. CI/CD

- **CI** (`ci.yml`): roda em todo PR para `homolog` ou `main`, com quatro verificações: lint de Conventional Commits (título do PR e commits), testes de backend (pytest, com migrations contra PostgreSQL), verificação do contêiner de cada serviço e testes de frontend (Jest). Ninguém mergeia com CI vermelho.
- **CD** (`deploy-fly.yml` e `deploy-vercel.yml`): o backend publica no push para `homolog` ou `main` que altere `src/backend/**`, rodando as migrations uma vez antes dos deploys e terminando com um teste de `GET /health` de cada serviço; o frontend publica só no push para `homolog`. Não há passo manual de deploy.
- **Ingestão da SEMOB** (`ingestao-semob.yml`): mensal, copia o catálogo de linhas e paradas para o banco.

Segredos e variáveis de cada plataforma, e a instalação local, estão nos guias `secrets-por-plataforma` e `setup-local` do repositório.

7. Definição de pronto (DoD), em dois níveis

A promoção para `main` costuma acontecer em lote, por isso uma US só fecha depois de dois níveis:

Nível	Critérios
DoD-Sprint (feita na sprint)	PR mergeado em `homolog` com aprovação formal e CI verde; cenários BDD validados em `homolog`; comentário na issue com o resultado
DoD-Release (entregue de verdade)	PR `homolog` → `main` aprovado e mergeado; validado em produção (tela ou endpoint real, não suposição); issue fechada; documentação atualizada, se a US alterou algo documentado

8. Regras para o uso de IA

- **O que o agente faz sem perguntar:** ler código, rodar testes locais, investigar bugs, criar e editar arquivos, criar branches, commitar e dar push em branches não protegidas, e consultar produção apenas para leitura (logs, variáveis, `curl`).
- **O que exige confirmação antes:** deletar recursos em produção, alterar produção (deploy manual, secrets, migrations contra o banco), mergear PR, bypass de proteção, editar issues de terceiros e qualquer ação fora do que foi pedido na conversa. Uma aprovação anterior não vale para uma ação nova.
- **Bypass de proteção** é decisão do usuário, nunca do agente, e todo bypass é registrado na issue (por quê, e que normalmente bloquearia o merge).
- **Verificar antes de declarar pronto:** ler a saída do comando, conferir o resultado real do CI (`gh pr checks`) e validar o endpoint depois do deploy. Quando o agente não pôde rodar algo, o relato diz isso ("testes escritos, não executados; validação pelo CI").
- **Divergência entre plano e código:** o agente expõe a divergência ao usuário e deixa a decisão explícita, em vez de escolher um lado em silêncio.

O que o processo ensinou

- **Aprovação por comentário não é aprovação** (issue #35): um "revisado" na conversa do PR foi confundido com review formal, e dali veio a regra de só contar o botão Approve.
- **Plano que diverge do código** (issue #35): o plano previa quatro apps e o código era um monolito; a divergência foi exposta e decidida por uma pessoa.
- **Diagnóstico por leitura de código nem sempre bate com a produção** (#213): o erro 500 nas preferências de notificação foi atribuído a uma violação de chave estrangeira; os logs do Fly mostraram outra causa (o serviço de Colaboração não carrega o modelo `Usuario`). Os testes passavam porque carregam todos os modelos juntos.
- **A causa era configuração, não código** (#216): o embarque aceito de longe levou a uma correção de código, mas o valor de `RAIO_EMBARQUE_M` em produção estava em 100 km. Antes de mexer no código de um comportamento que depende de limite, ler o valor efetivo no ambiente. Quando a causa não está confirmada, o PR usa `Refs`, não `Closes`.

9. Registrar as lacunas

Quando uma simplificação é feita para caber no prazo (como a falta de homologação separada), ela é documentada no lugar certo, comunicada ao usuário no momento da decisão e, se for risco antes de dado real, vira issue de acompanhamento. Débito técnico invisível é pior que débito técnico registrado.